

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 03-04/Unit 03: C Strings & Pointers

Tauqeer Saleem

Slides based on Dr Faisal Alvi's slides

For any suggested modifications please email: tauqeer.saleem@sse.habib.edu.pk



Unit Outline

1. C-Strings
2. Pointers
3. Pointers and Functions
4. Pointers and Arrays
5. Arrays, Pointers and Functions
6. Summary

C-Strings

A primitive implementation of strings in C language



C-Strings: Example Program

```
#include <iostream>

using namespace std;

int main(void) {
    const int MAX = 80;
    char str[MAX];

    cout << "Enter the string: ";
    cin >> str;

    cout << "The string you entered: " << str << endl;
    return 0;
}
```



C-Strings

C	O	M	P	U	T	E	R	\0	
---	---	---	---	---	---	---	---	----	--

- Each character occupies 1 byte of memory.
- C-strings must terminate with a byte containing 0.
- This is represented by the character constant `'\0'`, which has an ASCII value of 0. This terminating zero is called the null character.
- The `<<` operator displays characters until it encounters the null character.



C-Strings: Some pitfalls to avoid

- To avoid buffer overflow (what is a buffer?) while reading strings, use the `setw` operator. (Page 292, Lafore)
- You can initialize strings by directly assigning the string to the array (Page 292):

```
char str[] = "Hello! How are you?"
```

- To read text containing blanks using `cin`, we use another function, `cin.get()`.
- To read a string with multiple lines, use `cin::get()` with a terminating character.
- To copy a string character by character, use a for-loop.



C-Strings: Library Functions

- Some important C-String functions are given below.
- The #include file is cstring:

<u>strcat()</u>	Appends one C-style string to the end of another
<u>strcmp()</u>	Compares the ASCII values of characters in two C-style strings to determine which string has a higher value
<u>strcpy()</u>	Copies the characters of a C-style string into the memory of another string
<u>strlen()</u>	Return the length of a C-style string



Arrays of Strings

```
#include <iostream>
using namespace std;

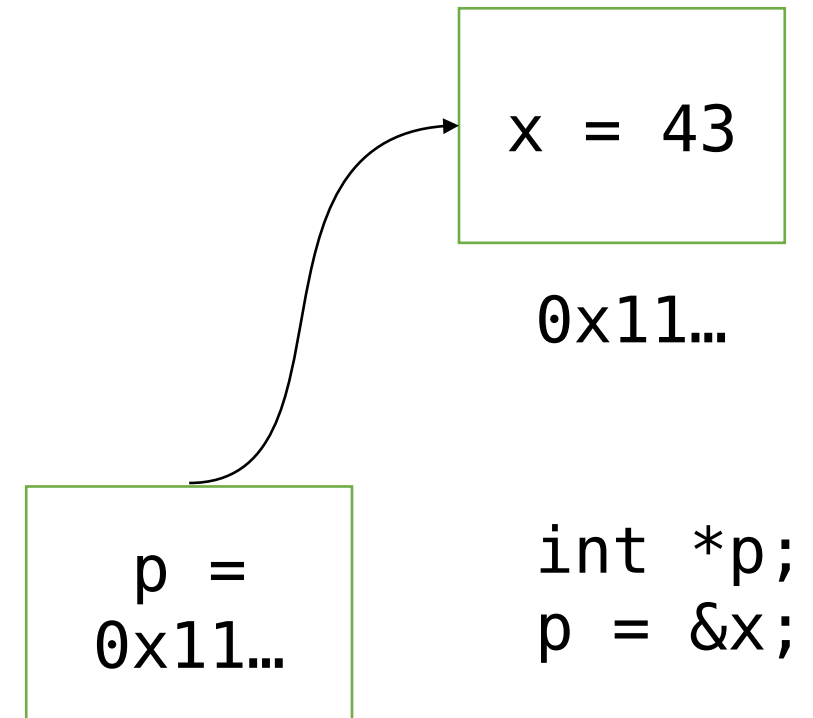
int main()
{
    const int DAYS = 7;           //number of strings in array
    const int MAX = 10;           //maximum size of each string
                                   //array of strings
    char star[DAYS][MAX] = { "Sunday", "Monday", "Tuesday",
                              "Wednesday", "Thursday",
                              "Friday", "Saturday" };
    for(int j=0; j<DAYS; j++)      //display every string
        cout << star[j] << endl;
    return 0;
}
```


Pointers

Cornerstone of C and C++

Pointers

- In C++, a pointer variable is an address to (or the memory location of) a variable.
- Let's say we define `int x = 43;`
- The variable `x` has a value of 43, and it has been allocated a memory location equal to 4 bytes.
- What is the actual memory address of `x`?
- This address can be founded (and printed) using a pointer variable.
- `int* p = &x;`





Pointers

- In C++, a pointer to a data type is declared as (type *).
- Therefore a pointer to an integer variable is declared as int*.
- Consider the following program →
- The output is:

```
x = 43
ptr = 0x61fe10
*ptr = 43
```

```
#include <iostream>
using namespace std;

int main(void) {
    int x = 43, y;
    int* ptr;

    ptr = &x;      Referencing
    y = *ptr;       Operator
                   Dereferencing
                   operator

    cout << "x = " << x << "\n";
    cout << "ptr = " << ptr << "\n";
    cout << "*ptr = " << y << "\n";

    return 0;
}
```



Pointer Variables – Declaration

- The operator '&' is the referencing operator. It links the pointer variable with the associated data value.
- The operator '*' is the dereferencing operator. It results in the value that is stored in the memory location stored within the pointer variable.
- Question: What is the output of: `int x = 4; cout << *(&x);`
- In C++, you need to declare the type of the variable the pointer is pointing to.
- So a pointer to a character is an address to a memory location having 1 byte, a pointer to an integer having 4 bytes and so on...
- In general, we can have `int* p`, `char* p`, `double* p` and so on.

Array-Pointer Duality

Arrays and Pointers can change role



Arrays and Pointers

- Recall that an array is a collection of homogeneous data types in C++.
- **The name of the array refers to the memory address of the first data item stored in the array.**
- In this sense, the **name of the array is** by itself, **an address** and can be treated as a pointer.
- Try the following code snippet:

```
int a[] = {5, 3, 4, 2, 1};  
cout << a;  
for(int ix = 0; ix < 5; ix++)  
    cout << *(a + ix) << endl;
```



Using an array name as a pointer (Ok)

```
// ptrnote.cpp
// array accessed with pointer notation
#include <iostream>
using namespace std;

int main()
{
    //array
    int intarray[5] = { 31, 54, 77, 52, 93 };

    for(int j=0; j<5; j++)
        cout << *(intarray+j) << endl;
    return 0;
}
```

- Can we use `*intarray++`?
- Why or why not?



Using an a pointer to an array (Ok)

```
// ptrinc.cpp
// array accessed with pointer
#include <iostream>
using namespace std;

int main()
{
    int intarray[] = { 31, 54, 77, 52, 93 }; //array
    int* ptrint;                             //pointer to int
    ptrint = intarray;                       //points to intarray

    for(int j=0; j<5; j++)                  //for each element,
        cout << *(ptrint++) << endl;       //print value
    return 0;
}
```


Pointers and Functions

Faster functions with pointers as arguments!

Pass by pointer vs pass by reference



Pointers And Function

- Typically variables are passed by value in C++ in functions.
- What does pass by value mean – it means a *copy* of the variable is passed to the function.
- Any modification to the passed parameter does not change the value of the argument.
- Consider the following program →
- The objective is to transfer the amount in the source to the dest. It doesn't work!

```
#include <iostream>
using namespace std;
void transfer(int, int);
```

```
int main(void) {
    int source = 4, dest = 0;
    cout << "Before the transfer: ";
    cout << source << ", " << dest << endl;
    transfer(source, dest);
    cout << "After the transfer: ";
    cout << source << ", " << dest << endl;
    return 0;
}
```

```
void transfer(int source, int dest) {
    dest = dest + source;
    source = 0;
}
```

```
//OUTPUT IS
//Before the transfer: 4, 0
//After the transfer: 4, 0
```



Pass By Pointers

- With pointers you can pass a value using its memory address.
- Pass by pointers means that, instead of passing the value, you now pass the *address* of that value.
- Any changes to the value pointed to by the address changes the value of the argument too.
- Consider the following program now →.
- Observe the syntax.

```
#include <iostream>
using namespace std;
void transfer(int* , int* );

int main(void) {
    int source = 4, dest = 0;
    cout << "Before the transfer: ";
    cout << source << ", " << dest << endl;
    transfer(&source, &dest);
    cout << "After the transfer: ";
    cout << source << ", " << dest << endl;
    return 0;
}

void transfer(int* source, int* dest) {
    *dest = *dest + *source;
    *source = 0;
}
```

```
//OUTPUT IS
//Before the transfer: 4, 0
//After the transfer: 0, 4
```



C++ Reference Variables

- A reference variable is an ***alias***, that is, another name for an already existing variable.
- Once a reference is initialized with a variable, either the variable name or the reference name may be used to refer to the variable.
- For example, consider

```
int a = 4; int &b = a;
```

- Here b and a refer to the same variable and hence b is a reference to a. Any changes to b reflect in a also.



Pass By Reference

- Another method for passing parameters in C++ is pass by reference.
- With this, we create a reference to the parameter that is supposed to be passed.
- Consider the following program now →.
- Observe the syntax.

```
#include <iostream>
```

```
using namespace std;
```

```
void transfer(int&, int&);
```

```
int main(void) {
```

```
    int source = 4, dest = 0;
```

```
    cout << "Before the transfer: ";
```

```
    cout << source << ", " << dest << endl;
```

```
    transfer(source, dest);
```

```
    cout << "After the transfer: ";
```

```
    cout << source << ", " << dest << endl;
```

```
    return 0;
```

```
}
```

```
void transfer(int& src, int& dst) {
```

```
    dst = dst + src;
```

```
    src = 0;
```

```
}
```

```
//OUTPUT IS
```

```
//Before the transfer: 4, 0
```

```
//After the transfer: 0, 4
```



Differences between references and pointers

- You cannot have NULL references. You must always be able to assume that a reference is connected to a legitimate piece of storage.
- Once a reference is initialized to an object, it cannot be changed to refer to another object. Pointers can be pointed to another object at any time.
- A reference must be initialized when it is created. Pointers can be initialized at any time.



Arrays, Pointers and Functions

- You have already seen how pointers can be used to pass addresses to individual variables.
- You may also pass a pointer to an array.
- However, it is recommended to pass the size of the array also as a parameter.
- Otherwise, this might lead to array decay – Explore – what is *array decay*?
- Note that arrays may also be passed by reference.
- Some examples follow.



```
int main() {  
    // Declare and initialize an array  
    int arr[5] = {1, 2, 3, 4, 5};  
  
    cout << "Before addOne: ";  
    for (int i = 0; i < 5; ++i) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
  
    // Call the function, passing the array by reference  
    addOne(arr);  
  
    // Print the array after modification  
    cout << "After addOne: ";  
    for (int i = 0; i < 5; ++i) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
  
    return 0;  
}
```

Passing an Array by Reference and by Pointer

```
// Function to add 1 to each element of array  
void addOne(int (&arr)[5]) { // Array reference  
    for (int i = 0; i < 5; ++i) {  
        arr[i] += 1;  
    }  
}
```

```
void addOne(int* arr, int size) {  
    for(int i = 0; i < size; i++) {  
        *(arr + i) += 1;  
    }  
}
```


Dynamic Memory Allocation

Faster functions with pointers as arguments!

Pass by pointer vs pass by reference



Static and Dynamic Arrays

- So far we have studied arrays that are initialized at compile time.
- For these arrays, the size has to be declared upfront.
- These automatically get deleted when the function defining them completes execution.
- These are known as static arrays and are allocated memory on the *stack*.
- Point to consider:
- What is a *stack* as an area of memory? What is a *heap*?
- Examples of static arrays:

```
int a[5]; char name[80];
```



Dynamic Array in C++

- In C++, dynamic arrays allow users to allocate memory dynamically.
- They are useful when the size of the array is not known at compile time.
- In C++, a dynamic array can be created using `new` keyword and can be deleted by using `delete` keyword.
- How to initialize a dynamic array?

```
int* arr = new int[n];
```

- A dynamic array is initialized at run time, on the *heap*, hence the name.
- Dynamic arrays have to be explicitly deallocated using the 'delete' keyword.



Dynamic Array in C++

+

```
#include <iostream>
using namespace std;

int main() {
    // Size of the array
    int n = 5;

    // Creating a dynamic array
    int* dynamicArray = new int[n];

    // Initializing the array
    for (int i = 0; i < n; ++i) {
        dynamicArray[i] = i + 1; // Assign values to the array
    }

    // Accessing the array
    for (int i = 0; i < n; ++i) {
        cout << dynamicArray[i] << " ";
    }

    // Deallocating the memory
    delete[] dynamicArray;

    return 0;
}
```

- Exploration Exercise:
- Explore what does the 'delete' keyword do?
- How does it work?
- Why is it necessary?

Double pointers

A pointer to a pointer?



Double Pointers

- Double Pointers in C++ can create a lot of confusion!
- In C++, double pointers are used, among other things, to refer to 2-D arrays.
- A 2-D array may simply be considered to be an array of arrays.
- These are stored in row-major order in main memory (What is row major order?)
- Consider the following:

```
int rows = 3, cols = 4;  
int** arr = new int*[rows];
```

- Declares an array of pointers!



Double Pointers - Example

```
int main() {
    int rows = 3, cols = 4;

    // Step 1: Declare a pointer to pointer
    int** arr = new int*[rows];

    // Step 2: Allocate memory for each row
    for (int i = 0; i < rows; ++i) {
        arr[i] = new int[cols];
    }

    // Step 3: Initialize the array
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            arr[i][j] = i + j; // Assign some values
        }
    }
}
```

```
// Step 4: Print the array
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < cols; ++j) {
        cout << arr[i][j] << " ";
    }
    cout << endl;
}

// Step 5: Free the allocated memory
for (int i = 0; i < rows; ++i) {
    delete[] arr[i]; // Delete each row
}
delete[] arr; // Delete the array of pointers

return 0;
}
```



Same Example with Pointer Arithmetics

```
int main() {
    int rows = 3, cols = 4;

    // Step 1: Declare a pointer to pointer
    int** arr = new int*[rows];

    // Step 2: Allocate memory for each row
    for (int i = 0; i < rows; ++i) {
        arr[i] = new int[cols];
    }

    // Step 3: Initialize the array using pointer ar
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            (*(arr + i) + j) = i + j;
            // Equivalent to arr[i][j]
        }
    }

    // Step 4: Print the array using pointer arithmetic
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            cout << (*(arr + i) + j) << " ";
            // Equivalent to arr[i][j]
        }
        cout << endl;
    }

    // Step 5: Free the allocated memory
    for (int i = 0; i < rows; ++i) {
        delete[] *(arr + i); // Free each row
    }
    delete[] arr; // Free the array of pointers

    return 0;
}
```




Double Pointers and Strings

```
int main() {  
    // Step 1: Declare an array of strings (array of char pointers)  
    const char* arr[] = {"Saturday", "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday"};  
  
    // Step 2: Declare a pointer to pointer (double pointer) to hold the array of strings  
    const char** ptr = arr;  
  
    // Step 3: Access the strings using the double pointer  
    cout << "First string: " << *ptr << endl;      // Prints "Saturday"  
    cout << "Second string: " << *(ptr + 1) << endl; // Prints "Sunday"  
    cout << "Third string: " << *(ptr + 2) << endl;  // Prints "Monday"  
  
    // Step 4: Access individual characters in the string using pointer arithmetic  
    cout << "First character of the second string: " << ***(ptr + 1) << endl; // Prints 'S'  
    cout << "Second character of the first string: " << **(*ptr + 1) << endl; // Prints 'S'  
  
    return 0;  
}
```